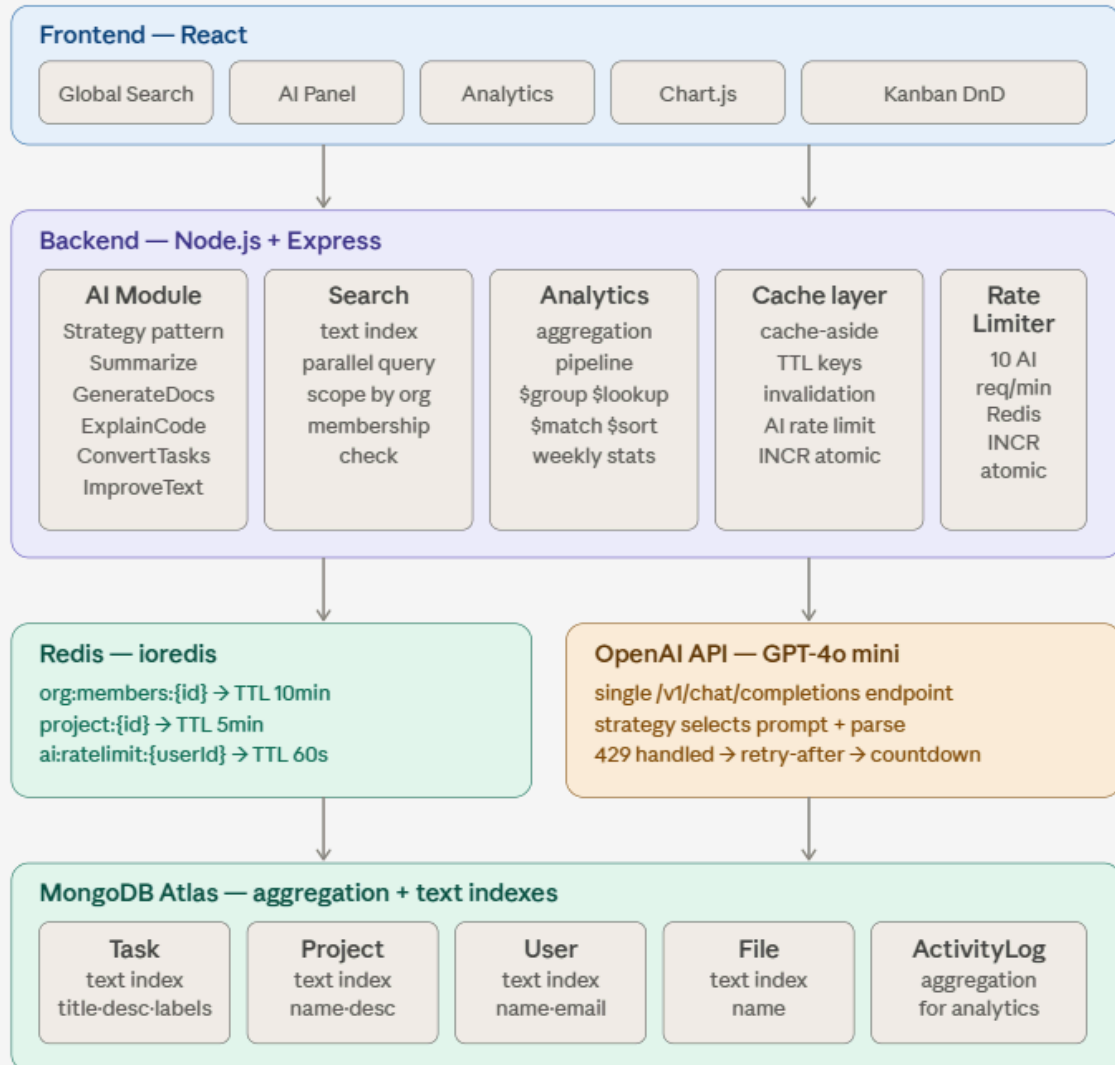
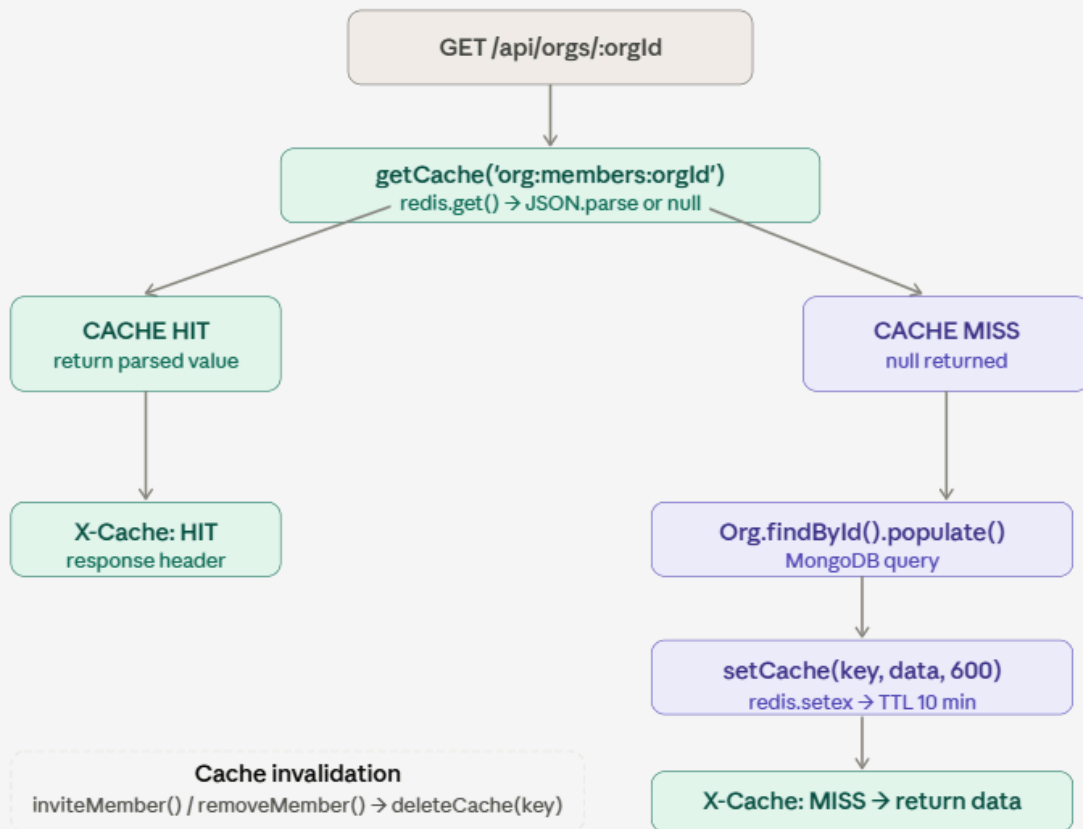


Yes, Phase 3 is 100% complete after all 7 prompts. Here's everything you need.

Phase 3 — complete system architecture



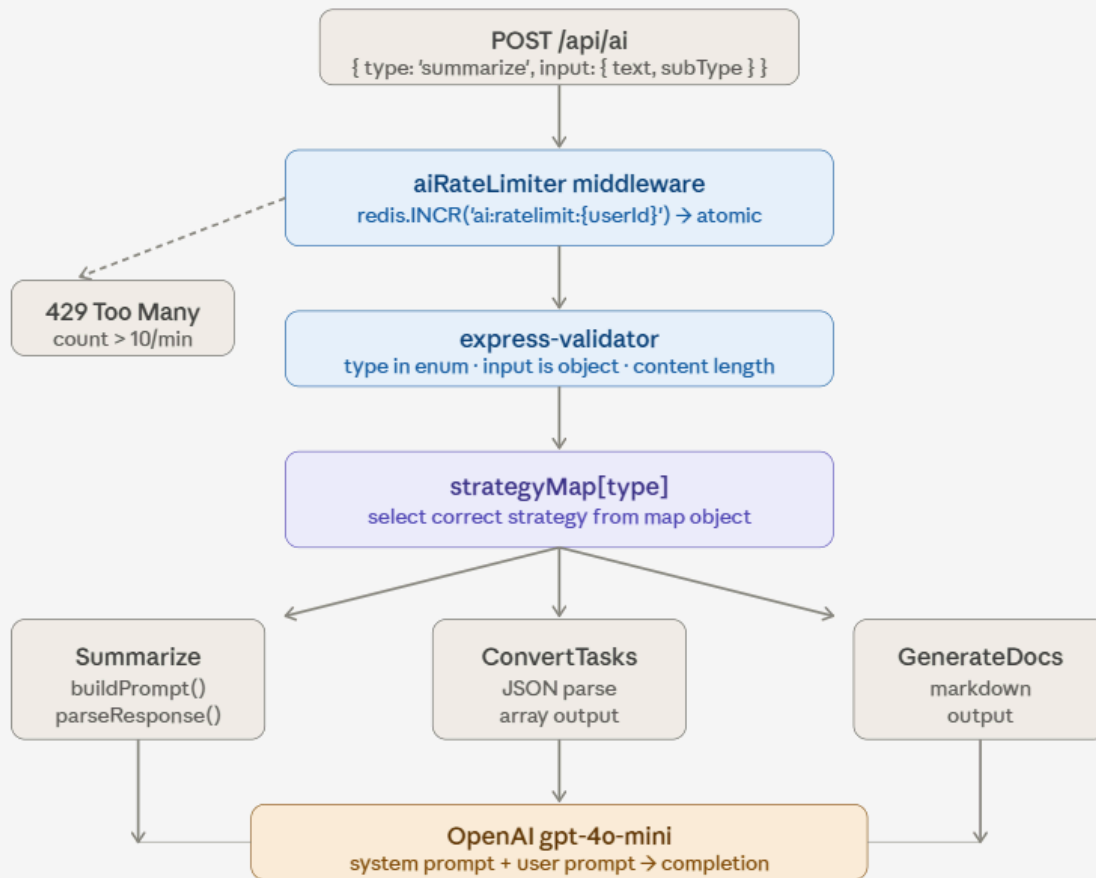
Flow 1 — Redis cache-aside pattern



Interview explanation: "I use the cache-aside pattern — the application code manages the cache, not the database. Every read checks Redis first. On a hit, I return immediately and set `X-Cache: HIT` in the response header. On a miss, I query MongoDB, store the result in Redis with a TTL, and set `X-Cache: MISS`. The key insight is invalidation — whenever a write mutates the cached data (invite member, update org), I explicitly call `deleteCache()` for that key. The next read repopulates it fresh. I chose TTLs deliberately: org members are stable, so 10 minutes. Notifications change rapidly, so 1 minute. The `X-Cache` header is visible in the dev tools Network tab — I built a `CacheIndicator` component that reads it and shows ⚡ `Cache HIT` in the corner during development."

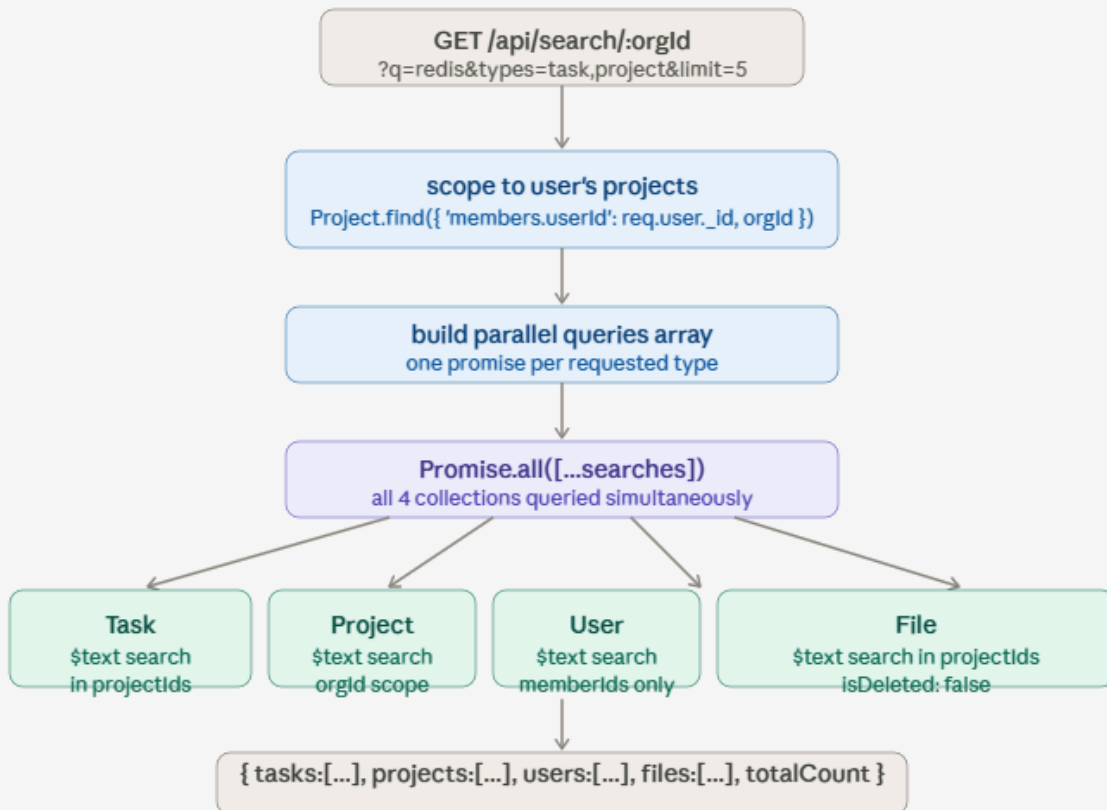


Flow 2 — AI Strategy pattern + rate limiting



Interview explanation: "The AI module uses the Strategy pattern. I have a `BaseAIStrategy` class with an `execute()` method. Each AI feature — summarize, explain code, generate docs, convert to tasks — is a subclass that overrides `buildSystemPrompt()`, `buildUserPrompt()`, and `parseResponse()`. The controller calls `strategyMap[type].execute(input)` — it never needs to know which strategy it's using. Adding a new AI feature means creating one new class and adding one entry to the map. Nothing else changes. The rate limiter uses Redis `INCR` — it increments a counter per user per minute. `INCR` is atomic in Redis, so concurrent requests don't race. On the first increment I set a 60-second TTL. If the counter exceeds 10, I return 429. This is per-user, not per-server — it works across multiple instances because Redis is shared."

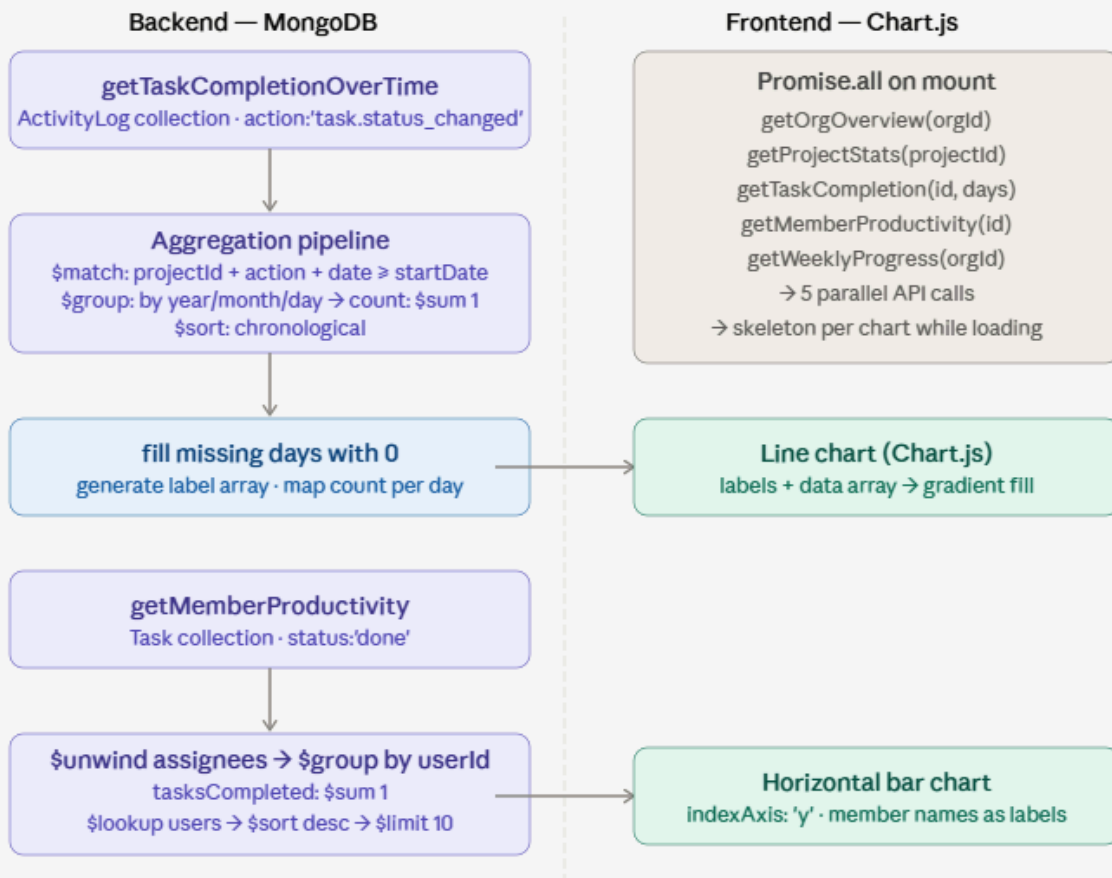
Flow 3 — Global search with parallel queries



Interview explanation: "Search works in three steps. First, I scope it — I get the IDs of every project the requesting user is a member of. This is the security boundary. A user can never see tasks from a project they don't belong to, even if they guess the project ID. Second, I run all four collection queries in parallel using `Promise.all` — tasks, projects, users, files all hit MongoDB simultaneously. This means search latency is the slowest single query, not the sum of all four. Third, MongoDB text indexes power the actual matching. I defined compound text indexes with weights — the task title gets weight 10, description gets weight 1, so a title match ranks higher than a body match. The `$text: { $search: query }` operator handles stemming and stop words automatically."



Flow 4 — Analytics aggregation pipeline



Interview explanation: "Analytics uses MongoDB's aggregation pipeline — a series of stages that transform documents. For task completion over time, I match the `ActivityLog` for status-change-to-done events, group by year/month/day using `$dayOfMonth` and `$month` operators, sort chronologically, then fill in missing days with zero on the application layer. This gives a clean array for Chart.js. For member productivity, I use `$unwind` on the assignees array — this creates one document per assignee per task — then group by userId summing completions, then `$lookup` to join user names. All five analytics queries fire in parallel with `Promise.all` on the dashboard mount. Each chart has its own loading skeleton, so charts pop in as each resolves rather than all waiting for the slowest one."

Phase 3 checklist

WEEK 6 — REDIS + AI

- ✓ **Redis singleton + cache utility**
getCache · setCache · deleteCache · deleteCachePattern · TTL constants
- ✓ **Cache key conventions**
org:members · project:{id} · ai:ratelimit:{userId} · notif:unread
- ✓ **Cache-aside on 4 services**
getOrgById · getMyOrgs · getProjectById · getUnreadCount
- ✓ **Cache invalidation**
deleteCache() on every write that mutates cached data
- ✓ **Socket.io Redis adapter**
createAdapter → events broadcast across server instances
- ✓ **AI rate limiter**
Redis INCR atomic · 10 req/min/user · expire 60s on first incr
- ✓ **BaseAIStrategy + 6 strategies**
Summarize · GenerateDocs · ExplainCode · ImproveText · ConvertTasks · README
- ✓ **Single AI endpoint**
POST /api/ai · type in body · strategyMap routing · OpenAI errors handled

WEEK 7 — SEARCH + ANALYTICS

- ✓ **MongoDB text indexes**
Task (title×10-labels×5-desc×1) · Project · User · File — all compound
- ✓ **Global search service**
membership scoping · Promise.all · 4 collections · limit per type
- ✓ **5 analytics endpoints**
orgOverview · projectStats · completionOverTime · memberProductivity · weeklyProgress
- ✓ **Aggregation pipelines**
\$match \$group \$unwind \$lookup \$sort — 5 distinct pipelines

PHASE 3 UI

- ✓ **GlobalSearch modal**
Cmd+K · keyboard nav · recent searches · highlight matches · 4 result types
- ✓ **AI Panel — 5 tabs**
Summarize · Generate · Explain · Improve · Convert with full UX per tab
- ✓ **AI loading + error states**
animated thinking dots · rate limit countdown timer · retry button
- ✓ **Convert to tasks flow**
AI generates array · add individual · add all · progress counter · success flash
- ✓ **Analytics dashboard**
line chart · donut · grouped bar · horizontal bar · stat cards · real data
- ✓ **Chart.js dark theme**
global defaults · TOOLTIP_CONFIG · SCALE_CONFIG · gradient fill on line
- ✓ **Kanban drag and drop rebuilt**
pointer fix · edge auto-scroll · StatusChangeBadge · land-flash animation
- ✓ **Cache indicator (dev)**
X-Cache header · HIT=green · MISS=amber · auto-hide 3s · dev only

The 10 questions interviewers will ask about Phase 3

"What is the cache-aside pattern and why did you choose it over write-through?"

In cache-aside, the application code manages the cache — it checks Redis first, falls back to the DB on miss, then populates the cache. In write-through, every DB write also writes to cache. I chose cache-aside because my data has asymmetric access patterns — reads are frequent, writes are rare. Write-through would cache data that's never read again, wasting Redis memory. Cache-aside also degrades gracefully — I wrapped every Redis call in try/catch, so if Redis is down, the app falls back to MongoDB transparently.

"How do you handle cache invalidation — the hardest problem in CS?"

I use explicit invalidation — when a write mutates cached data, I call `deleteCache()` immediately. For example, `inviteMember()` deletes `org:members:{orgId}`. The next read repopulates it. I chose delete-on-write over update-on-write because it's simpler and correct — there's no risk of the cache holding stale data from a failed partial update. The tradeoff is one extra DB read after each write, which is acceptable given how infrequently writes happen.

"Why is Redis INCR the right choice for rate limiting?"

`INCR` is atomic in Redis — it increments a value and returns the new count in a single operation. Without atomicity, two concurrent requests could both read `count=9`, both decide they're under the limit, and both proceed — you'd get 11 requests through. With `INCR`, only one thread gets `count=10` and the next gets 11, correctly rejected. I set the TTL on the first increment only, so the window resets naturally after 60 seconds.

"How does MongoDB text search work under the hood?"

MongoDB maintains an inverted index — a mapping from each word to the documents containing it. When I run `$text: { $search: 'redis caching' }`, MongoDB splits the query into tokens, looks each up in the inverted index, takes the union of matching documents, scores them by relevance using the weights I defined (`title×10`, `description×1`), and returns them sorted by score. The compound text index covers all searchable fields in one index — MongoDB only allows one text index per collection.

"Why does your search scope results to user's projects?"

Security. Without scoping, a user could search for tasks across the entire database by guessing common words. Every search first fetches the array of project IDs where the requesting user is a member — this is their visibility boundary. The task query then adds `projectId: { $in: projectIds }` as a hard filter. Even if MongoDB's text search matches a document, it won't be returned if it's in a project the user can't access.

"Explain the aggregation pipeline for member productivity."

Five stages. `$match` filters to done tasks in the project. `$unwind` on the assignees array explodes each task into one document per assignee — a task with three assignees becomes three documents. `$group` by `userId` sums the count. `$lookup` joins the users collection to get

names and avatars. `$sort` descending by count, `$limit 10`. The key insight is `$unwind` — without it you can't aggregate on array elements individually.

"What is the Strategy pattern and where exactly is it in your code?"

Strategy defines a family of algorithms, encapsulates each one, and makes them interchangeable. In my AI module, `BaseAIStrategy` defines the interface — `buildSystemPrompt()`, `buildUserPrompt()`, `parseResponse()`, and `execute()`. Each AI feature is a concrete strategy. The `strategyMap` object maps the `type` string to the right instance. The controller calls `strategyMap[type].execute(input)` — it's completely decoupled from which strategy runs. The Open/Closed Principle applies directly: adding a new AI feature means creating one new file and one new map entry, zero changes to existing code.

"How does the drag and drop pointer fix work?"

The original issue was spreading `draggableProps` and `dragHandleProps` on different elements — one on the wrapper, one on a child. `@hello-pangea/dnd` calculates the drag offset relative to the element that receives `dragHandleProps`. When they're on different elements, the offset calculation is wrong and the card snaps away from the cursor. The fix is spreading both props on the same outermost div. The `getDraggableStyle` function then adds the rotation and shadow without overriding the library's transform calculation.

"How does the edge auto-scroll work during drag?"

I wrote a `useDragScroll` hook that starts a `requestAnimationFrame` loop when drag begins. On every mouse move, I check the cursor's distance from the board container's left and right edges. If within 120px, I calculate a scroll speed proportional to how close the cursor is — near the edge means faster scroll. The speed is applied to `boardRef.current.scrollLeft` each frame. This works independently of `@hello-pangea/dnd`'s internal mechanics, so it doesn't interfere with the drop position calculation.

"What would you add to the analytics if you had more time?"

Three things. First, real-time chart updates — the dashboard currently fetches on mount and on project change, but a socket event when a task is completed could increment the `completedThisWeek` counter without a full refetch. Second, custom date ranges — currently fixed at 7/14/30 days, but a date picker would give more flexibility. Third, export to CSV — the aggregation data is already structured, adding a CSV serializer and download endpoint would take maybe two hours. I'd mention these as future enhancements in the interview.

The design patterns used — summary for whiteboard

Pattern	Where	What it solves
Strategy	AI module strategies SummarizeStrategy etc.	Swap AI behavior without changing the controller
Observer	EventBus listeners notify + log on events	Decouple task actions from side effects (Phase 2)
Singleton	Redis client, DB conn, OpenAI client	One connection, reused everywhere
Cache-aside	org.service getOrgById project.service etc.	Read Redis first, populate on miss, delete on write
Factory	strategyMap object { summarize: new S() }	Map type string to correct strategy instance
Decorator	Express middleware chain on AI route	authenticate → rateLimiter → validator → controller

The most impressive thing you can say in an interview: "Every design decision in Phase 3 has a specific reason. Redis TTLs are different per data type based on volatility. Parallel search uses `Promise.all` because the queries are independent. The Strategy pattern means adding a seventh AI feature takes one file and one line. These weren't defaults — they were deliberate choices I can defend."